Advance Searching Algorithms

WEEK 8

6-10/10/2025

Shell Sort

- Insertion Sort builds a sorted subarray one element at a time.
- Each new element is compared with elements in the sorted part and inserted at the correct position.

- Shell Sort is an in-place comparison-based sorting algorithm.
- It is basically an improved version of Insertion Sort.
- The main problem with Insertion Sort is that it moves elements one step at a time.
- Shell Sort overcomes this by allowing the exchange of far apart elements first, reducing the total number of shifts needed later.

Working Principle:

- Choose a **gap sequence** (common choice: n/2, n/4, …, 1).
- For each gap:
  - Treat the array as gap separate sub-arrays.
  - Apply **insertion sort** to each sub-array.
- Reduce the gap and repeat until the gap = 1.
- When gap = 1, the array is nearly sorted → insertion sort becomes very fast.

Shell Sort

Characteristics:

- **In-place algorithm** (no extra memory).
- **Not stable** (relative order of equal elements may not be preserved).
- **Time Complexity:**
  - Best: O(n log n)
  - Average: O(n^1.5) approx (depends on gap sequence)
  - Worst: $O(n^2)$
- **Space Complexity:** O(1)

Why Faster than Insertion Sort?

- In Insertion Sort, if the smallest element is at the end, it must be swapped through the entire array (n-1 moves).
- In Shell Sort, with a large gap, that element can move to the front in fewer steps.
- This reduces the number of moves dramatically.

Shell Sort

- **Algorithm Steps**
- Choose a **gap sequence** (commonly gap = n/2, then gap/2, until gap = 1).
- For each gap, perform a **gapped insertion sort**:
  - Compare elements that are gap distance apart.
  - Swap if needed.
- Keep reducing the gap until it reaches **1**.
- At gap = 1, the list will be almost sorted, so the final pass is efficient.

Shell Sort

Example (Dry Run)

- Let's sort this array:
  arr = [23, 12, 1, 8, 34, 54, 2, 3]

Step 1: n = 8, initial gap = 4

- Compare elements 4 apart.

arr[0] vs arr[4] → 23 vs 34 (ok)

arr[1] vs arr[5] → 12 vs 54 (ok)

arr[2] vs arr[6] → 1 vs 2 (ok)

arr[3] vs arr[7] → 8 vs 3 → swap → [23,12,1,3,34,54,2,8]

Step 2: gap = 2

- Now check elements 2 apart.

Compare arr[0] vs arr[2] → 23 vs 1 → swap → [1,12,23,3,34,54,2,8]

Compare arr[1] vs arr[3] → 12 vs 3 → swap → [1,3,23,12,34,54,2,8]

Compare arr[2] vs arr[4] → 23 vs 34 (ok)

Compare arr[3] vs arr[5] → 12 vs 54 (ok)

Compare arr[4] vs arr[6] → 34 vs 2 → swap → [1,3,23,12,2,54,34,8]

Compare arr[5] vs arr[7] → 54 vs 8 → swap → [1,3,23,12,2,8,34,54]

Shell Sort

- **Step 3: gap = 1**

(normal insertion sort on nearly sorted array)

- [1,3,23,12,2,8,34,54]

→ insert 23 ok

→ insert 12 → [1,3,12,23,2,8,34,54]

→ insert 2 → [1,2,3,12,23,8,34,54]

→ insert 8 → [1,2,3,8,12,23,34,54]

Final sorted array = [1, 2, 3, 8, 12, 23, 34, 54]

Shell Sort --- Code

```
void shellSort(int arr[], int n) {
    // Start with a big gap, then reduce the gap
    for (int gap = n/2; gap > 0; gap /= 2) {
        // Do a gapped insertion sort for this gap size
        for (int i = gap; i < n; i++) {
            int temp = arr[i];
            int j;
            // Shift earlier gap-sorted elements up until the correct location for arr[i] is found
            for (j = i; j >= gap && arr[j - gap] > temp; j -= gap) {
                arr[j] = arr[j - gap];
            }
            // Put temp (the original arr[i]) in its correct location
            arr[j] = temp;
        }
    }
}
```

Complexity Analysis

- **Best case:** O(n log n) (when array is already sorted or nearly sorted).
- **Worst case:** $O(n^2)$ (but better than insertion sort in practice).
- **Average case:** Between O(n log n) and O(n^1.5), depending on gap sequence.
- **Space Complexity:** O(1) (in-place).

Radix Sort

- Radix Sort is a non-comparative sorting algorithm.
- It sorts numbers digit by digit, starting from the least significant digit (LSD) to the most significant digit (MSD) (or sometimes vice versa).
- It uses a stable sorting algorithm (like Counting Sort) as a subroutine to sort based on each digit.

What is Counting Sort?
Counting Sort is a **non-comparison sorting algorithm**. Instead of comparing elements (like Bubble, Quick, or Merge Sort), it **counts how many times each element occurs**.
It is very efficient when:

The range of input numbers (k) is not much larger than the number of elements (n).

It works only on **integers or things that can be mapped to integers** (like characters).

Radix Sort

Steps of Radix Sort (LSD Method)

- Find the maximum number in the array (to know the number of digits).
- Set **exp = 1** (for units place).
- Use **Counting Sort** to sort elements according to the current digit (at position exp).
- Multiply exp *= 10 to move to the next digit (tens, hundreds, etc.).
- Repeat until the most significant digit is processed.

Radix Sort

- **Example (Dry Run)**

arr = [170, 45, 75, 90, 802, 24, 2, 66]

Step 1: Maximum = 802 (3 digits) → so we process 3 rounds.

Round 1 (exp = 1 → unit place):

- Sort by last digit:
- 170(0), 90(0), 802(2), 2(2), 24(4), 45(5), 75(5), 66(6)
- → [170, 90, 802, 2, 24, 45, 75, 66]

Round 2 (exp = 10 → tens place):

- Sort by tens digit:
- 802(0), 2(0), 24(2), 45(4), 66(6), 170(7), 75(7), 90(9)
- → [802, 2, 24, 45, 66, 170, 75, 90]

Radix Sort

Round 3 (exp = 100 → hundreds place):

- Sort by hundreds digit:
- 2(0), 24(0), 45(0), 66(0), 75(0), 90(0), 170(1), 802(8)

→ [2, 24, 45, 66, 75, 90, 170, 802]

Final sorted array: [2, 24, 45, 66, 75, 90, 170, 802]

Radix Sort (Bucket Style Dry Run)

- **Round 1 → exp = 1 (Units digit)**

We look at the **last digit** of each number and place them in buckets 0–9.

Buckets (by units place)	Contents
0	170, 90
1	
2	802, 2
3	
4	24
5	45, 75
6	66
7–9	—

After collecting from buckets **in order**:

```
[170, 90, 802, 2, 24, 45, 75, 66]
```

Radix Sort (Bucket Style Dry Run)

- **Round 2 → exp = 10 (Tens digit)**
- Now we look at the **second last digit** (tens).

Buckets (by tens place)	Contents
0	802, 2
1	
2	24
3	
4	45
5	
6	66
7	170, 75
8	
9	90

After collecting:

```
[802, 2, 24, 45, 66, 170, 75, 90]
```

Radix Sort (Bucket Style Dry Run)

- **Round 3 → exp = 100 (Hundreds digit)**
- Now we look at the **third digit** (hundreds).

Buckets (by hundreds place)	Contents
0	2, 24, 45, 66, 75, 90
1	170
2–7	—
8	802
9	—

Final Sorted Array

```
[2, 24, 45, 66, 75, 90, 170, 802]
```

Radix Sort

```
// Get maximum value in array
int getMax(int arr[], int n) {
    int mx = arr[0];
    for (int i = 1; i < n; i++)
        if (arr[i] > mx)
            mx = arr[i];
    return mx;
}

// Radix
```

Explanation of implementation:

- We use **buckets logic(0–9)**.
- For each digit place:
  - Distribute numbers into buckets based on that digit.
  - Collect them back in order.
- Repeat until all digits are processed.

```
//Sort implementation (LSD)
void radixSort(int arr[], int n) {
    int maxVal = getMax(arr, n);
    // exp = 1 (units), 10 (tens), 100 (hundreds) ...
    for (int exp = 1; maxVal / exp > 0; exp *= 10) {
        // Buckets for digits 0-9
        vector<int> bucket[10];
        // Place elements into buckets based on current digit
        for (int i = 0; i < n; i++) {
            int digit = (arr[i] / exp) % 10;
            bucket[digit].push_back(arr[i]);
} // Collect elements back into array
        int idx = 0;
        for (int d = 0; d < 10; d++) {
            for (int val : bucket[d]) {
                arr[idx++] = val;
            }       }   }    }
```

Radix Sort

- Another way to implement **Radix Sort (LSD version)** by using **Counting Sort** as a subroutine:
- Code on GCR

Quick Sort

- Quick Sort is a divide and conquer sorting algorithm.
- It works by choosing a pivot element, partitioning the array so that:
  - Elements smaller than pivot go to the left
  - Elements greater than pivot go to the right
- Then recursively sort the left and right subarrays.

Steps of Quick Sort

- **Choose a pivot** (commonly the last element).
- **Partition** the array:
  - Reorder elements so that all smaller go before pivot, and all larger after.
  - Place pivot in its correct sorted position.
- **Recurse** on left and right subarrays.
- Combine results.

Quick Sort- Dry run

- Array:

[10, 80, 30, 90, 40, 50, 70]

Pivot = 70
Partition → [10, 30, 40, 50] [70] [80, 90]

Left Subarray = [10, 30, 40, 50]
Pivot = 50
Partition → [10, 30, 40] [50] []

[10, 30, 40]
Pivot = 40
Partition → [10, 30] [40] []

[10, 30]
Pivot = 30
Partition → [10] [30] []

Sorted:

[10] [30] [40] [50] [70] [80] [90]

Quick Sort- Code

```
// Function to swap two elements
void swap(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}
// Partition function
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // choosing last element as pivot
    int i = low - 1;       // index of smaller element
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
```

```
    // place pivot in correct position
    swap(arr[i + 1], arr[high]);
    return (i + 1);
}

// QuickSort function
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        // Partition index
        int pi = partition(arr, low, high);

        // Recursively sort elements before and after
partition
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

Quick Sort- **Partition Function Dry Run**

- **Call:**

partition(arr, 0, 6) // arr[0..6], pivot = arr[6] = 70

pivot = 70

i = -1 (start before the first element)

We loop j from low (0) to high-1 (5).

- **Iteration 1: j = 0**

arr[j] = 10, which is < pivot (70)

Increment i = 0, swap arr[i] with arr[j] → no change.

Array: [10, 80, 30, 90, 40, 50, 70]

- **Iteration 2: j = 1**

arr[j] = 80, which is > 70

Do nothing. Array unchanged.

Iteration 3: j = 2

arr[j] = 30, which is < 70

Increment i = 1, swap arr[i] (80) with arr[j] (30).
Array: [10, 30, 80, 90, 40, 50, 70]

Iteration 4: j = 3
arr[j] = 90, which is > 70
Do nothing.

Iteration 5: j = 4
arr[j] = 40, which is < 70
Increment i = 2, swap arr[i] (80) with arr[j] (40).
Array: [10, 30, 40, 90, 80, 50, 70]

Iteration 6: j = 5
arr[j] = 50, which is < 70
Increment i = 3, swap arr[i] (90) with arr[j] (50).
Array: [10, 30, 40, 50, 80, 90, 70]

Final Step: Place pivot
Now loop ends.
Swap pivot (70) with arr[i+1] = arr[4].
Array becomes:
[10, 30, 40, 50, 70, 90, 80]

Quick Sort- **Partition Function Dry Run**

- Partition is complete:
- Left side: [10, 30, 40, 50] (all < 70)
- Pivot: [70] (correct position)
- Right side: [90, 80] (all > 70)
- Next, Quick Sort will recursively sort [10, 30, 40, 50] and [90, 80].

Remaining Sorting Steps

- We already did this earlier. Result:

Left: [10, 30, 40, 50] (index 0–3)

Right: [90, 80] (index 5–6)

- **Step 2: Left Subarray [10, 30, 40, 50]**

quickSort(arr, 0, 3)

Pivot = 50.

Partition → [10, 30, 40, 50, 70, 90, 80]

(no swaps needed except placing pivot).

Pivot index = **3**. Split into:

Left: [10, 30, 40] (0–2) Right: [] (none).

- **Step 3: Subarray [10, 30, 40]**

quickSort(arr, 0, 2)

Pivot = 40.

Partition → [10, 30, 40, 50, 70, 90, 80]

Pivot index = **2**.Split into:

Left: [10, 30] (0–1) Right: []

Step 4: Subarray [10, 30]
quickSort(arr, 0, 1)
Pivot = 30. Partition →
[10, 30, 40, 50, 70, 90, 80]
Pivot index = 1.
Split into:
Left: [10]
Right: []
Both are size 1 → sorted.
Left part is now fully sorted:
[10, 30, 40, 50]

Step 5: Right Subarray [90, 80]
quickSort(arr, 5, 6)
Pivot = 80. Partition →
Swap 90 ↔ 80 →
[10, 30, 40, 50, 70, 80, 90]
Pivot index = 5.
Split into:
Left: []
Right: [90]
Sorted.

Final Result
The array is now fully sorted:
[10, 30, 40, 50, 70, 80, 90]

Quick Sort

- **Time Complexity**
- **Best case:** $O(n \log n)$ → Balanced partitions
- **Average case:** $O(n \log n)$
- **Worst case:** $O(n^2)$ → Already sorted array with bad pivot selection
- **Space Complexity:** $O(\log n)$ (recursive stack)

Merge Sort

- **Merge Sort** is a **Divide and Conquer** sorting algorithm.
- It divides the array into two halves, sorts each half, and **merges** them back together.
- It is **stable** (preserves order of equal elements).

Working Steps:

- **Divide:**
  Split the array into two halves (until only one element remains).
- **Conquer:**
  Recursively sort both halves.
- **Combine:**
  Merge the two sorted halves into a single sorted array.

Merge Sort

Property	Description
Algorithm type	Divide & Conquer
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	O(n log n)
Space Complexity	O(n) (needs temporary arrays)
Stable	Yes
In-place	No (requires extra space)

Merge Sort – Dry Run

Example Array

- A = [38, 27, 43, 3, 9, 82, 10] Indexes : 0 1 2 3 4 5 6
- mergeSort(A, 0, 6)

STEP 1: Splitting Phase

mergeSort(A, 0, 6)

- mid = (0 + 6) / 2 = 3

Split → Left: (0–3) and Right: (4–6)

- Left: [38, 27, 43, 3]
- Right: [9, 82, 10] **Sort Left Half → mergeSort(A, 0, 3)**
- mid = (0 + 3) / 2 = 1
- Split → Left: (0–1), Right: (2–3)
- **Left side:** [38, 27] **Right side:** [43, 3]

Sort [38, 27] → mergeSort(A, 0, 1)

- mid = 0
- Split → (0–0) and (1–1)
- Now each side has **one element**, so we merge them.
- Compare step by step:
- 38 > 27 → put 27 first then 38
- Result: [27, 38]

Sort [43, 3] → mergeSort(A, 2, 3)

- mid = 2
- Split → (2–2) and (3–3)→ [43] and [3]
- Now merge them.

Merge (2–2, 3–3)

- Left: [43]
- Right: [3]
- Compare:
- 43 > 3 → put 3 first, then 43
- Result: [3, 43]

Merge both halves of left side → merge(A, 0, 1, 3)

Left: [27, 38]

Right: [3, 43]

- Compare step-by-step:
- 27 > 3 → put 3
- 27 < 43 → put 27
- 38 < 43 → put 38
- Copy leftover 43

Result: [3, 27, 38, 43]

So left half (0–3) sorted

Now sort Right Half (4–6) → mergeSort(A, 4, 6)

- mid = (4 + 6) / 2 = 5
- Split → Left: (4–5), Right: (6–6)
- Left side: [9, 82] Right side: [10]

Sort Left → mergeSort(A, 4, 5)

- mid = 4
- Split → (4–4) and (5–5) → [9] and [82]
- **Merge (4–4, 5–5)**
- Left: [9]
- Right: [82]
- 9 < 82 → put 9
- Then 82
- Result: [9, 82] **Now merge [9, 82] with [10] → merge(A, 4, 5, 6)**
- Left: [9, 82]
- Right: [10]
- Compare:
- 9 < 10 → put 9
- 82 > 10 → put 10
- Copy leftover 82
- Result: [9, 10, 82]

right half (4–6) sorted ✓

STEP 2: Final Merge (0–3) and (4–6)

- Now merge both sorted halves:
- Left: [3, 27, 38, 43]
- Right: [9, 10, 82]

Step	Compare (L vs R)	Smaller	Placed in Array	New Array
1	3 vs 9	3	A[0] = 3	[3,27,38,43,9,10,82]
2	27 vs 9	9	A[1] = 9	[3,9,38,43,9,10,82]
3	27 vs 10	10	A[2] = 10	[3,9,10,43,9,10,82]
4	27 vs 82	27	A[3] = 27	[3,9,10,27,9,10,82]
5	38 vs 82	38	A[4] = 38	[3,9,10,27,38,10,82]
6	43 vs 82	43	A[5] = 43	[3,9,10,27,38,43,82]
7	L exhausted	copy 82	A[6] = 82	[3,9,10,27,38,43,82]

Final array:
[3, 9, 10, 27, 38, 43, 82]

Merge Sort -- Code

```
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];
    // Copy data
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];
    int i = 0, j = 0, k = left;
    // Merge two halves
    while (i < n1 && j < n2) {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }
```

```
    // Copy remaining elements
    while (i < n1)
        arr[k++] = L[i++];
    while (j < n2)
        arr[k++] = R[j++];
}
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
```

Summary

1. Quick Sort

- **Best for:** Large datasets where average performance matters.
- Very fast in practice (less overhead).
- Works **in-place** (no extra array memory).
- Average Time = **O(n log n)**
- **Weakness:**
- Worst case = **$O(n^2)$** (if pivot chosen poorly, e.g., already sorted data).
- Not stable (equal elements may reorder).
- **Use it when:** you want high speed and memory efficiency, and you can tolerate instability.

2. Merge Sort

- **Best for:** Data that must remain **stable** (relative order preserved).
- Always **O(n log n)** (worst, best, and average).
- Works well with **linked lists** (no extra array copying).
- Good for **external sorting** (like sorting data on disk).
- **Weakness:**
- Needs extra **O(n)** space.
- More overhead for small arrays.
- **Use it when:** stability or guaranteed O(n log n) time matters more than memory.

Summary

3. Shell Sort

- **Best for: Medium-sized arrays** or when simple implementation is needed.
- Faster than insertion sort for medium n.
- Works **in-place** (no extra memory).
- **Weakness:**
- Not stable.
- Performance depends on the gap sequence (average ≈ O(n^1.5)).
- **Use it when:** array is small/medium and you want a simple, in-place algorithm.

4. Radix Sort

- **Best for:** Numbers or fixed-length strings.
- Linear time **O(n ⋅k)** (k = number of digits).
- Works great for integers or IDs.
- **Weakness:**
- Needs extra memory (buckets).
- Doesn't work well for comparison-based sorting (like floats, structs, etc.).
- **Use it when:** sorting integers, phone numbers, or ZIP codes.

Comparison Summary Table

Algorithm	Best Case	Avg Case	Worst Case	Space	Stable	Suitable For
Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n)	No	Large datasets
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	Linked lists, stable sort
Shell Sort	O(n log n)	O(n^1.5)	O(n²)	O(1)	No	Medium arrays
Radix Sort	O(n·k)	O(n·k)	O(n·k)	O(n + k)	Yes	Integer/character data

Scenario	Recommended Sorting
Large dataset, average speed needed	**Quick Sort**
Guaranteed performance, stable needed	**Merge Sort**
Integers or fixed-size keys	**Radix Sort**
Small/medium array, simple code	**Shell Sort**

"Quick Sort" is fastest on average.
"Merge Sort" is safest and stable.
"Radix Sort" is best for numbers.
"Shell Sort" is simple and efficient for small arrays.